

**Московский авиационный институт
(национальный исследовательский университет)**

**Факультет информационных технологий и прикладной
математики**

Кафедра вычислительной математики и программирования

Лабораторные работы по курсу «Информационный поиск»

Студент: А. А. Останина
Преподаватель: А. А. Кухтичев
Группа: М8О-408Б-22
Дата:
Оценка:
Подпись:

Москва, 2025

Содержание

1	Добыча и анализ корпуса	2
2	Поисковый робот	4
3	Токенизация	6
4	Закон Ципфа	8
5	Стемминг	8
6	Булев индекс	10
7	Булев поиск	11
8	Выводы	13

1 Добыча и анализ корпуса

Постановка задачи

Необходимо проанализировать корпус документов, который будет использован при выполнении остальных лабораторных работ:

- Скачать примеры документов к себе на компьютер. В отчёте нужно указать источник данных. Источников в итоговом индексе должно быть не менее двух;
- Ознакомиться с ним, изучить его характеристики. Из чего состоит текст? Есть ли дополнительная мета-информация? Если разметка текста, какая она?
- Выделить текст.
- Найти существующие поисковики, которые уже можно использовать для поиска по выбранному набору документов (встроенный поиск Википедии, поиск Google с использованием ограничений на URL или на сайт). Если такого поиска найти невозможно, то использовать корпус для выполнения лабораторных работ нельзя!
- Привести несколько примеров запросов к существующим поисковикам, указать недостатки в полученной поисковой выдаче.

В результатах работы должна быть указаны статистическая информация о корпусе:

- Размер примеров «сырых» документов.
- Количество документов.
- Размер текста, выделенного из «сырых» данных.
- Средний размер документа, средний объём текста в документе.

Описание и результаты

Корпус собирался из двух источников: [Wikipedia](#) и [Culture.ru](#).

В качестве тематического направления была выбрана классическая музыка: композиторы, произведения, музыкальные жанры, театры и исполнители. Для Wikipedia отбирались релевантные статьи (по категориям и навигационным связям внутри музыкального раздела), после чего скачивалась HTML-страница каждой статьи. Для Culture.ru использовались материалы портала (карточки персоналий, статьи/заметки, связанные с музыкой и культурным наследием), также в виде исходного HTML.

На этапе предобработки из «сырых» HTML-документов извлекался основной текст. Для каждого документа в корпусе фиксировались:

- нормализованный URL страницы;
- источник (Wikipedia или Culture.ru);
- время скачивания (Unix timestamp);
- исходный HTML и выделенный из него текст.

Примеры существующих поисковиков/способов поиска по источникам:

- **Wikipedia:** встроенный поиск по сайту (строка поиска на ru.wikipedia.org) позволяет находить статьи по ключевым словам и перенаправлениям.
- **Culture.ru:** встроенный поиск портала (строка поиска на culture.ru) поддерживает поиск по материалам сайта.
- **Google (ограничение по сайту):** поиск по конкретному источнику через запросы вида `site:ru.wikipedia.org «классическая музыка»` или `site:culture.ru «серебряный век»`.

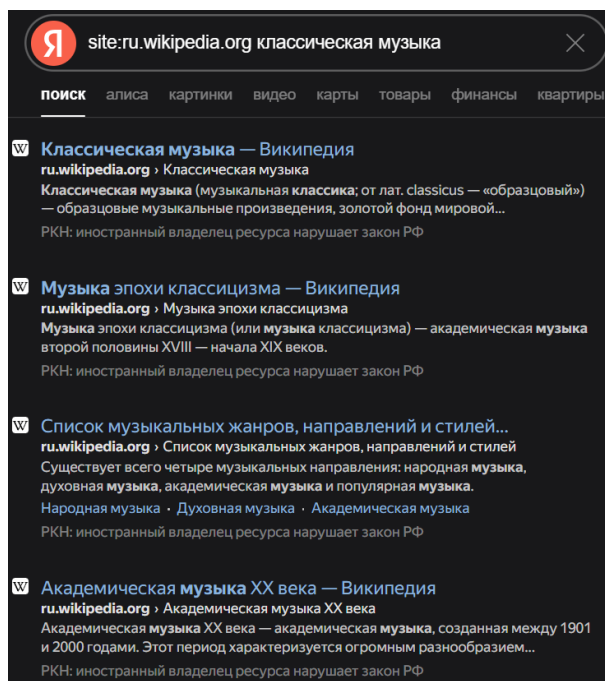


Рис. 1: Пример встроенного поиска по Wikipedia

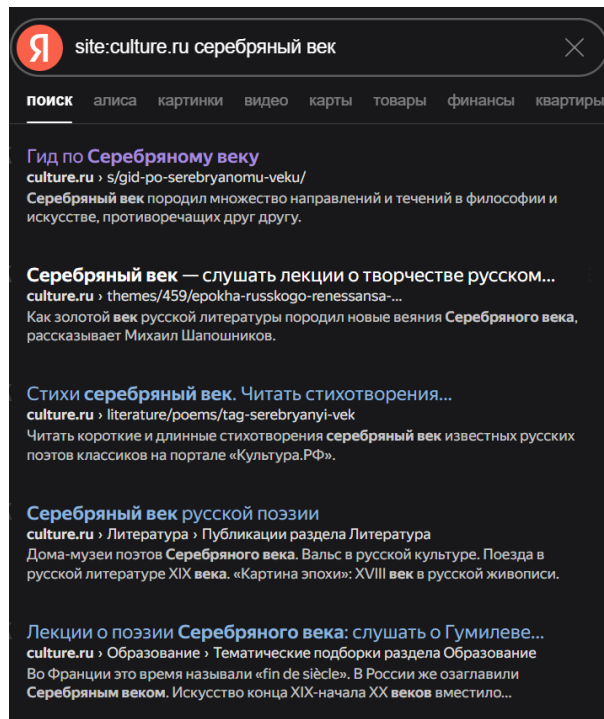


Рис. 2: Пример встроенного поиска по portalу Culture.ru

Статистика корпуса (извлечение текста):

Параметр	Значение
Количество документов	90 740
Общий размер текста	2052.84 MB
Средний размер текста документа	23 722 байт
Средняя длина документа	13 702 символа
Время извлечения текста	644.39 сек
Скорость извлечения	3262.18 KB/сек

2 Поисковый робот

Постановка задачи

Необходимо написать парсер на любом языке программирования.

- Написать поисковый робот — компоненты обкачки документов, используя любой язык программирования;

- Единственным аргументом поисковому роботу подаётся путь до yaml-конфига, содержащий:
 - Данные для базы данных в секции db;
 - Данные для робота в секции logic: задержка между обкачкой страницы;
 - Любые другие данные, необходимые для реализации логики поискового робота.
- Сохранять в базе данных (например, MongoDB) документы со следующими полями:
 - url, нормализованный;
 - «сырой» html-текст документа;
 - название источника;
 - Дата обкачки документа в формате Unix time stamp.
- Поисковый робот можно остановить в любой момент и при повторном запуске робот должен начать с того документа, с которого он остановился;
- Периодически он должен уметь переобкачивать документы, которые уже есть в базе, но только в том случае, если они изменились.

Описание реализации

Поисковый робот систематически обходит веб-страницы, извлекает их содержимое и сохраняет в базу данных MongoDB для последующего анализа.

Основная логика работы робота сосредоточена в классе **SearchRobot**. При запуске он загружает конфигурацию из YAML-файла, который определяет параметры подключения к базе данных, настройки обкачки (задержки, таймауты, количество потоков), а также список источников для сканирования с их правилами фильтрации.

Робот начинает работу с подключения к MongoDB, где создает необходимые индексы для оптимизации запросов. Затем он загружает свое предыдущее состояние — список уже обработанных URL и очередь на скачивание, что позволяет корректно возобновить работу после остановки. Это реализовано через сохранение очереди в отдельную коллекцию **crawler_state** и восстановление оттуда при перезапуске.

Важной частью является нормализация URL, которая выполняется классом **URLNormalizer**. Он приводит все адреса к единому формату, удаляя фрагменты, приводя к нижнему регистру и обрезая слешы, что позволяет избежать дублирования страниц из-за различий в написании одного и того же URL. Фильтрация URL осуществляется классом

`URLFilter`, который проверяет каждый адрес по белым и черным спискам регулярных выражений, определенным для каждого источника в конфигурации.

Основной цикл работы построен на многопоточности. Робот создает пул потоков, каждый из которых независимо обрабатывает URL из общей очереди. Для каждого URL проверяется, не скачивался ли он ранее и не пришло ли время для повторной проверки (согласно интервалу `recheck_interval`). Если страница уже есть в базе и не изменилась, робот просто обновляет время последней проверки, экономя ресурсы. При скачивании используется механизм повторных попыток и задержки между запросами, чтобы избежать блокировки со стороны серверов.

После успешной загрузки страницы робот анализирует её HTML-содержимое, извлекая все ссылки с помощью регулярных выражений. Найденные ссылки нормализуются, фильтруются и добавляются в очередь для последующей обработки. Сами страницы сохраняются в коллекцию MongoDB вместе с метаданными: исходным URL, сырым HTML, названием источника и временем скачивания в формате Unix timestamp.

При повторном скачивании страницы робот вычисляет хеш-сумму HTML и сравнивает с сохраненной. Если контент не изменился, он лишь обновляет время последней проверки, что снижает нагрузку на сеть и базу данных. Для управления работой робота предусмотрена корректная обработка сигналов остановки (`Ctrl+C`), при которой происходит сохранение текущего состояния очереди перед завершением.

3 Токенизация

Постановка задачи

Нужно реализовать процесс разбиения текстов документов на токены, который потом будет использоваться при индексации. Для этого потребуется выработать правила, по которым текст делится на токены. Необходимо описать их в отчёте, указать достоинства и недостатки выбранного метода. Привести примеры токенов, которые были выделены неудачно, объяснить, как можно было бы поправить правила, чтобы исправить найденные проблемы.

В результатах выполнения работы нужно указать следующие статистические данные: количество токенов, среднюю длину токена.

Кроме того, нужно привести время выполнения программы, указать зависимость времени от объёма входных данных. Указать скорость токенизации в расчёте на килобайт входного текста. Является ли эта скорость оптимальной? Как её можно ускорить?

Описание реализации

Токенизатор реализован на языке C++ (`common/src/tokenizer.cpp`) в виде класса `Tokenizer`.

Алгоритм обработки:

1. **Первичное разбиение:** Текст делится на фрагменты по любому количеству пробельных символов с использованием регулярного выражения `S+`.
2. **Очистка границ (`cleanToken`):** Из начала и конца каждого выделенного фрагмента удаляются знаки пунктуации. *Важное правило:* Дефис внутри слова не считается пунктуацией и сохраняется для корректной обработки сложных слов (например, «рок-н-ролл»). Однако на данном этапе очищаются только ASCII-символы пунктуации (с кодом < 128).
3. **Приведение к нижнему регистру (`toLower`):** Для латиницы используется стандартное смещение. Для кириллицы реализована побитовая обработка: извлекается двухбайтовый код символа, и если он попадает в диапазон заглавных букв русского алфавита (`0x0410-0x042F`), к нему прибавляется смещение `0x20`. Отдельно обрабатывается буква «Ё».

Преимущества и недостатки:

- **Плюсы:** Высокая скорость (более 8 МБ/сек), отсутствие внешних зависимостей, корректная работа с базовым русским алфавитом.
- **Минусы:**
 - Метод `S+` является достаточно агрессивным и может захватывать нежелательные символы в середине токена.
 - Метод очистки границ не обрабатывает UTF-8 знаки пунктуации (например, французские кавычки «*«»*» или специфические тире), так как проверяет только символы с кодом меньше 128.

Примеры неудачной токенизации:

- «музыка — открывающая кавычка-елочка остается в токене, так как она является многобайтовым символом UTF-8 и не обрабатывается функцией `cleanToken`.
- `автор.права` — если точка находится внутри токена без окружающих пробелов, она не будет удалена, так как очистка производится только по краям.

Возможные улучшения: Для исправления проблем можно было бы использовать таблицы категорий Unicode для определения любых знаков пунктуации вне зависимости от их кодировки. Также использование более сложных регулярных выражений на базе свойств символов (`\p{L}`) позволило бы сразу выделять буквенные последовательности.

Результаты

Параметр	Значение
Всего токенов	166 573 599
Уникальных токенов	5 403 920
Время выполнения	249.16 сек
Скорость (по тексту)	8 436.79 КВ/сек

4 Закон Ципфа

Постановка задачи

Для своего корпуса необходимо построить график распределения терминов по частотностям в логарифмической шкале, наложить на этот график закон Ципфа. Объяснить причины расхождения.

Результаты

Показатель аппроксимации $\alpha \approx 1.3437$. Расхождения на концах графика связаны с большим количеством редких слов (имена собственные, опечатки, артефакты токенизации) и спецификой кириллического корпуса.

5 Стемминг

Постановка задачи

Добавить в созданную поисковую систему лемматизацию/стемминг. В простейшем случае, это просто поиск без учёта словоформ. В более сложном случае, можно давать бонус большего размера за точное совпадение слов.

Лемматизацию можно добавлять на этапе индексации, можно на этапе выполнения поискового запроса. В отчёте должна быть включена оценка качества поиска, после

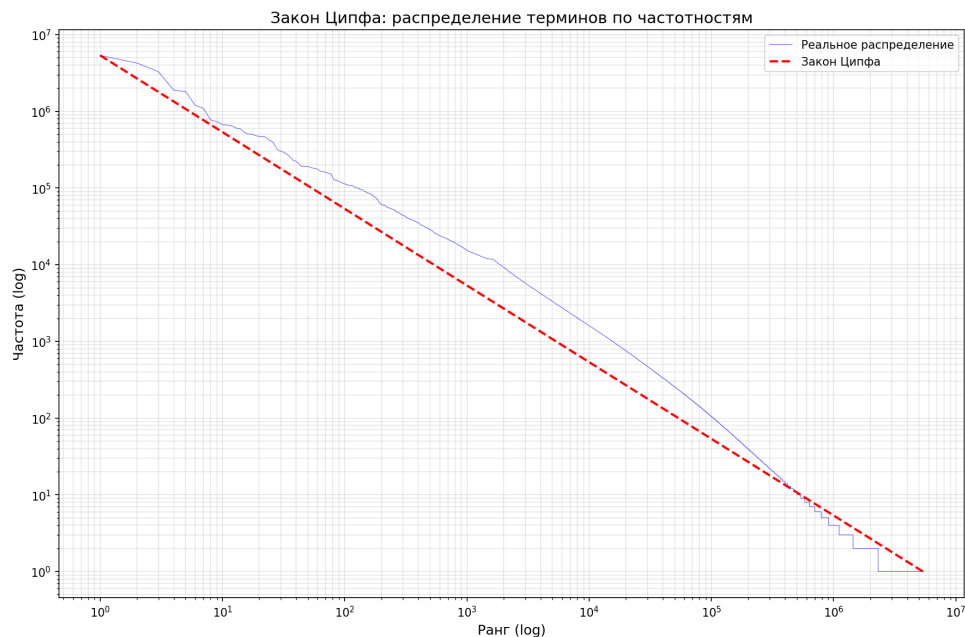


Рис. 3: Распределение Ципфа для собранного корпуса

внедрения лемматизации. Стало ли лучше? Изучите запросы, где качество ухудшилось. Объясните причину ухудшения и как можно было бы улучшить качество поиска по этим запросам, не ухудшая остальные запросы?

Описание реализации

Реализован алгоритм стемминга Портера для русского языка (Snowball) на языке C++. Этот алгоритм является стеммером, основанным на правилах (rule-based), который последовательно удаляет или заменяет окончания и суффиксы для выделения основы слова.

Алгоритм и области слова: Для корректной работы стеммер выделяет в слове три области:

- **RV:** Область после первой гласной в слове. Большинство окончаний удаляются именно в этой зоне.
- **R1:** Область после первой комбинации «гласная + согласная».
- **R2:** Область после первой комбинации «гласная + согласная» в зоне R1.

Основные шаги стемминга:

1. **Шаг 1:** Поиск и удаление окончаний причастий, деепричастий, рефлексивных окончаний («-ся», «-сь»), прилагательных и глаголов в зоне RV.
2. **Шаг 2:** Удаление окончания «и», если оно присутствует.
3. **Шаг 3:** Поиск и удаление деривационных суффиксов (например, «-ост», «-ость») в зоне R2.
4. **Шаг 4:** Удаление мягкого знака, суффиксов превосходной степени («-ейш», «-ейше») и двойных согласных «nn».

Результаты

Общая статистика стемминга:

Параметр	Значение
Уникальных слов до стемминга	5 403 920
Уникальных основ после стемминга	4 457 462
Процент измененных токенов	53.07%
Время выполнения фильтрации	326.88 сек

Примеры объединения словоформ: Стемминг позволяет объединить десятки различных грамматических форм одного слова в единую основу:

Основа	Кол-во форм	Примеры исходных форм
созда	92	создал, создала, созданный, создавшая, создадим
занима	86	занимал, занимаемая, занимаемся, занимающийся
счита	85	считается, считали, считанного, считавшая
называ	85	называемый, называлась, называвшихся, называя

Использование стемминга сократило размер словаря на 17.5%, что существенно повышает эффективность индексации и полноту поиска.

6 Булев индекс

Постановка задачи

Написать программу на C++ для построения булевого индекса по корпусу документов. Должна быть возможность выбора использовать стемминг или нет.

Описание реализации

Процесс индексации реализован в виде отдельного модуля (`search_engine/src/index.cpp`), который преобразует набор термов в оптимизированную структуру для быстрого поиска.

Процесс построения индекса:

1. **Сбор пар:** Из корпуса извлекаются пары (терм, ID документа).
2. **Сортировка:** Весь массив пар сортируется сначала по терму, а затем по ID документа. Использован **итеративный QuickSort**, что критично для обработки миллионов записей, так как предотвращает переполнение стека (Stack Overflow). Выбор пивота методом «медианы из трех» и разделение Хоара обеспечивают стабильную производительность порядка $O(N \log N)$.
3. **Агрегация:** После сортировки идентичные термы объединяются, а их уникальные DocID группируются в отсортированные списки (`postings lists`).

Бинарный формат индекса (IDX1): Индекс сохраняется в бинарном файле для минимизации объема и мгновенной загрузки в память:

- **Заголовок:** Магическая сигнатура IDX1 (4 байта) и количество уникальных термов в словаре.
- **Словарь термов:** Для каждого терма записывается его длина, строковые данные, количество связанных документов (`docFreq`) и сами списки DocID. Списки документов записываются сразу за записью термина, что позволяет считывать их последовательно.

Поиск по словарю в загруженном индексе осуществляется методом бинарного поиска, что обеспечивает скорость доступа к списку документов за $O(\log N)$.

7 Булев поиск

Постановка задачи

Написать программу на C++ для осуществления булевого поиска с использованием булевого индекса. Сравнить время поиска.

Описание реализации

Поисковая система (`search_engine/src/search_engine.cpp`) реализует обработку сложных логических выражений с использованием классических алгоритмов обработки выражений и оптимизированных операций над множествами.

Процесс выполнения запроса:

1. **Препроцессинг и неявный AND:** Запрос очищается, приводится к нижнему регистру и стеммится. Для удобства пользователя реализована автоматическая вставка оператора **AND** между термами, если между ними нет явного оператора (например, `опера музыка` интерпретируется как `опера AND музыка`).
2. **Алгоритм Shunting-yard:** Инфиксная запись запроса (со скобками и приоритетами) преобразуется в Обратную польскую нотацию (RPN). Установлены следующие приоритеты операторов: **NOT** (3) — наивысший, **AND** (2), **OR** (1) — наинизший.
3. **Вычисление RPN:** Результирующее выражение вычисляется с использованием стека результатов.

Оптимизация операций над множествами: Так как все списки DocID в индексе отсортированы, базовые операции пересечения (**AND**), объединения (**OR**) и разности (**NOT**) выполняются методом двух указателей за линейное время $O(n + m)$.

Механизм «ленивой инверсии» (Lazy Negation): Реализация оператора **NOT** напрямую (как создание списка всех DocID, кроме заданных) неэффективна для больших корпусов. В системе реализован флаг `isNegated` в объекте результата. Это позволяет вычислять сложные цепочки:

- `A AND NOT B` вычисляется как `difference(A, B)`.
- `NOT A AND NOT B` вычисляется как `NOT union(A, B)` (согласно законам де Моргана).
- `NOT A OR NOT B` вычисляется как `NOT intersection(A, B)`.

Такой подход позволяет избегать работы с огромными списками «всех документов корпуса» вплоть до финального этапа выдачи.

Примеры поисковых запросов: Ниже приведены примеры выполнения поисковых запросов к разработанной системе:

```

(venv) anna@LAPTOP-932TP917:~/labs_7sem/InfoSearch/search_engine$ ./search_engine.out search_index_stemmed.bin ../data/corpus.txt.urls --stemming
Loading URLs from ../data/corpus.txt.urls...
Loaded 90740 URLs.
Loading Index from index_stemmed.bin...
Index loaded in 0.975664s.
Index Statistics:
Dictionary Size: 4457462 terms
Total Postings: 69043251

Enter boolean queries (e.g., 'term1 AND term2'). Type 'exit' to quit.
> музыка
Found 18671 documents in 0.12963ms.
Results (first 5 and last 5):
[1] https://ru.wikipedia.org/wiki/классическая_музыка
[2] https://ru.wikipedia.org/wiki/%d0%92%d0%b5%d1%80%d0%b4%d0%b8_%d0%94%d0%b6%d1%83%d0%b7%d0%b5%d0%b7%d0%b7%d0%b5
[3] https://ru.wikipedia.org/wiki/%d0%a4%d0%b6%d1%80%d1%82%d0%b5%d0%b7%d0%b8%d0%b0%d0%b0%d0%b0
[4] https://ru.wikipedia.org/wiki/%d0%90%d0%b0%d0%b5%d0%ba%d1%81%d0%b0%d0%b0%d0%b4%d1%80_%d0%b0%d0%b5%d0%b7%d1%81%d0%ba%d0%b8%d0%b9_(%d1%84%d0%b8%d0%bb%d1%8c%d0%bc)
[5] https://ru.wikipedia.org/wiki/%d0%92%d0%b0%d0%b3%d0%b0%d0%b5%d1%80_%d0%a0%d0%b8%d1%85%d0%b0%d1%80%d0%b4
...
[90617] https://www.culture.ru/poems/34321/maska-muzyka
[90664] https://www.culture.ru/events/5375261/programma-muzykalno-literaturnaya-gostinaya
[90721] https://www.culture.ru/institutes/36554/galereya-proun
[90736] https://www.culture.ru/poems/47841/katyusha
[90739] https://www.culture.ru/poems/47818/parennyok

```

Рис. 4: Пример поискового запроса 1

```

> серебряный AND век
Found 24 documents in 0.115812ms.
Results (first 5 and last 5):
[7543] https://ru.wikipedia.org/wiki/%d0%90%d1%81%d0%bb%d0%b0%d0%b0%d0%b4%d0%b8%d1%8f
[10046] https://ru.wikipedia.org/wiki/%d0%9a%d0%bb%d0%b8%d0%bc%d1%82_%d0%93%d1%83%d1%81%d1%82%d0%b0%d0%b2
[11973] https://ru.wikipedia.org/wiki/%d0%9f%d1%83%d1%81%d1%81%d0%b5%d0%bd_%d0%9d%d0%b8%d0%ba%d0%b0%d0%bb%d0%b0
[15917] https://ru.wikipedia.org/wiki/%d0%a%d1%84%d0%b8%d0%b0%d0%b7%d0%b8%d1%8f
[18700] https://ru.wikipedia.org/wiki/%d0%a1%d0%bb%d0%b0%d0%b2%d1%8f%d0%b0%d1%81%d0%ba%d0%b0%d1%8f_%d0%bc%d0%b8%d1%84%d0%b0%d0%bb%d0%be%d0%b3%d0%b8%d1%8f
...
[58666] https://ru.wikipedia.org/wiki/%d0%90%d0%bc%d1%83%d0%b0%d0%b4%d1%81%d0%b5%d0%bd_%d0%a0%d1%83%d0%b0%d0%bb%d1%8c
[61429] https://www.culture.ru/materials/255481/istoriya-starinnykh-zagadok
[61821] https://www.culture.ru/materials/151775/zhizn-knyazya-vladimira-v-zerkale-knizhnoi-miniatury
[62428] https://www.culture.ru/materials/151775/zhizn-knyazya-vladimira-v-zerkale-knizhnoy-miniatury
[76411] https://www.culture.ru/institutes/9453/daurskii-gosudarstvennyi-prirodnyi-biosfernyi-zapovednik

```

Рис. 5: Пример поискового запроса 2

```

> мозикл OR опера
Found 20559 documents in 0.143584ms.
Results (first 5 and last 5):
[1] https://ru.wikipedia.org/wiki/классическая_музыка
[2] https://ru.wikipedia.org/wiki/%d0%92%d0%b5%d1%80%d0%b4%d0%b8_%d0%94%d0%b6%d1%83%d0%b7%d0%b5%d0%b7%d0%b7%d0%b5
[4] https://ru.wikipedia.org/wiki/%d0%90%d0%bb%d0%b5%d0%ba%d1%81%d0%b0%d0%b0%d0%b4%d1%80_%d0%b0%d0%b5%d0%b7%d1%81%d0%ba%d0%b8%d0%b9_(%d1%84%d0%b8%d0%bb%d1%8c%d0%bc)
[5] https://ru.wikipedia.org/wiki/%d0%92%d0%b0%d0%b3%d0%b0%d0%b5%d1%80_%d0%a0%d0%b8%d1%85%d0%b0%d1%80%d0%b4
[8] https://ru.wikipedia.org/wiki/%d0%9a%d0%b5%d0%bb%d0%b4%d1%80%d1%88_%d0%a0%d1%80%d0%b8%d0%b9_%d0%92%d1%81%d0%b5%d0%b7%d0%b0%d0%bb%d0%be%d0%b0%d0%b0%d0%b0%d0%b0
...
[90486] https://www.culture.ru/events/549127/muzikl-ostorozhno-deti
[90508] https://www.culture.ru/live/movies/46324/iskusstvo-v-emigracii-mark-shagal
[90537] https://www.culture.ru/afisha/sverdlovskaya-oblast-ekaterinburg
[90592] https://www.culture.ru/live/movies/621/zvezdnaya-pyl
[90720] https://www.culture.ru/institutes/12024/memorialnaya-kvartira-svyatoslava-rikhtera

```

Рис. 6: Пример поискового запроса 3

8 Выводы

В ходе лабораторных работ была разработана модульная и масштабируемая система поиска, способная эффективно работать с многоязычными запросами. Были реализованы и исследованы ключевые компоненты поисковой системы: сбор и индексация документов, токенизация, стемминг и булев поиск. Практическая проверка подтвердила применимость закона Ципфа для собранного корпуса. В результате создан работоспособный поисковый движок, демонстрирующий понимание основных алгоритмов информационного поиска.